四、预训练时代

在前面的演进过程中，我们已经看到，从 RNN 到 Seq2Seq，再到 Attention，模型逐渐从“局部建模”走向“全局依赖建模”。但在这一阶段，Attention 仍然主要作为 RNN 的增强模块存在，并没有改变整个建模范式。

真正的转折点发生在 2017 年。

1. Transformer 横空出世

2017 年，Google 团队发表论文《Attention is All You Need》。

这篇论文提出了一个非常重要的思想：

> **完全抛弃 RNN 和 CNN，仅依赖 Attention 来构建整个序列建模模型。**

也就是说，它不再使用传统的“循环结构”或“卷积结构”，而是用 Attention 直接完成序列之间的依赖建模。在结构上，它仍然属于 Encoder–Decoder 框架，但与之前不同的是：

- Encoder 不再使用 RNN，而是由多层 Attention + 前馈网络组成
- Decoder 同样由 Attention 模块堆叠构成
- 序列之间的关系完全由 Attention 机制建模

因此，Transformer 本质上可以理解为：

> **一个完全基于 Attention 构建的序列建模框架**

这一设计使得模型可以在训练时实现高度并行，同时仍然能够建模全局依赖关系，因此迅速成为 NLP 领域的核心架构。

1.1 整体概述

1.1.1 Transformer 到底指什么?

在实际使用中，“Transformer”这个词经常被简化使用，因此容易让初学者产生一个误解：

> Transformer 只是某一个模块或某一种注意力结构。

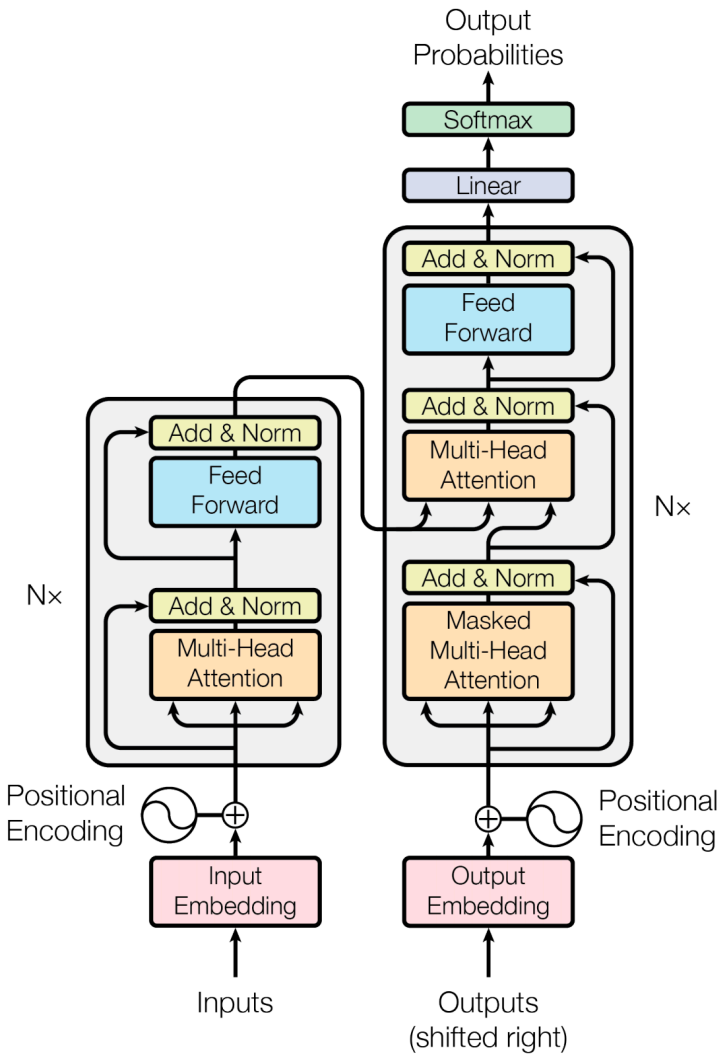
但实际上，Transformer 指的是一套**完整的序列建模架构体系**，而不是单一组件。

从结构层面来看，Transformer 是一类基于 Attention 的序列建模架构，既可以采用 Encoder–Decoder 结构，也可以扩展为 Encoder-only 或 Decoder-only 形式。其核心目标是：

> **建模输入序列与输出序列之间的全局依赖关系。**

1.1.2 整体结构

Transformer整体结构如下图所示：



从整体上看，Transformer 可以抽象为如下结构：

```
输入序列
    ↓
Encoder（多层堆叠）
    ↓
中间表示（Context）
    ↓
Decoder（多层堆叠）
    ↓
输出序列
```

在这个框架中，Encoder 和 Decoder 内部并不是单一操作，而是由多个功能模块组合而成。

主要包括：

- Attention 机制（核心信息交互方式）

- Feed Forward Network（特征变换）
- Positional Encoding（位置信息建模）
- Residual Connection（稳定训练）

需要注意的是，这些模块并不是“并列关系”，而是共同构成了 Transformer 的基本计算单元。

可以这样理解 Transformer：

> Transformer = “用 Attention 作为核心计算方式的 Encoder–Decoder 架构”

接下来，我们将从 Encoder 的内部结构开始，逐层拆解 Transformer 是如何工作的。

1.2 Encoder模块：输入预处理

1.2.1 主要步骤

在进入 Encoder 之前，文本需要先转换成模型可以处理的形式，主要包含三个步骤：

（1）文本切分与编号

首先，将输入文本拆分成 token（可以是单词或子词），然后转换为数字 ID，方便模型计算。

```
我 喜欢 机器学习
→ 12 58 1034 76
```

（2）长度对齐

由于每个句子的长度不同，为了能够批量训练，需要将短句补齐到统一长度。

```
短句：我 喜欢
补齐：我 喜欢 [PAD] [PAD]
```

（3）**忽略补齐部分**

模型在计算时会通过注意力掩码（attention mask）忽略填充部分，只关注真实文本部分。

（4）处理结果

经过上述处理，输入文本会被转换为一个三维张量表示：`(batch_size, seq_len, hidden_dim)`

其中：

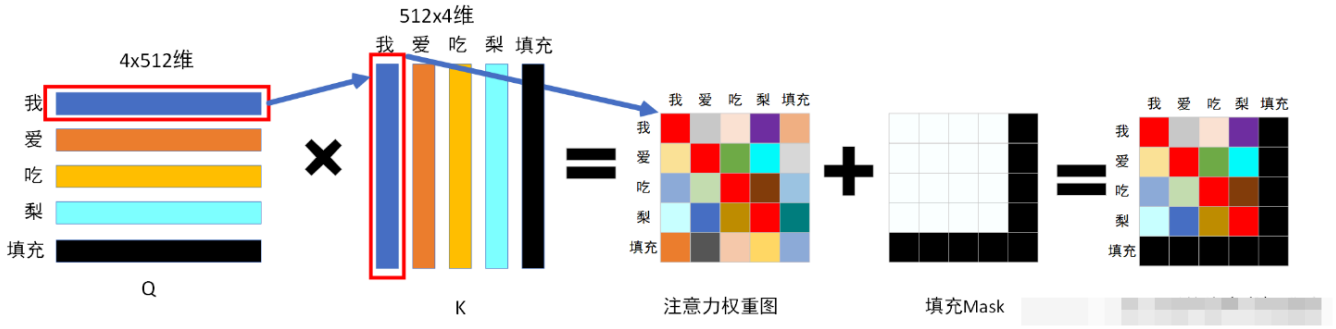
- batch_size 表示一次输入的句子数量
- seq_len 表示每个句子的 token 数量（已对齐）

- hidden_dim 表示每个 token 对应的向量维度（语义空间维度）

也就是说，模型的输入不再是“文本序列”，而是一个“带有维度结构的语义矩阵”。

1.2.2 注意力掩码

在深度学习中的注意力机制里，注意力掩码是一种关键的技术，用于控制模型在计算注意力时忽略不相关的部分。



注意力掩码的主要作用是屏蔽掉不相关的部分，使得模型在计算注意力时只关注有效的信息。Softmax 使用指数函数 e^x，所以即使输入的注意力分数是 0，经过 e^0=1 后，它依然会分得一定的权重概率，而不是变成 0。

这就意味着，**如果不加干预，模型永远无法真正做到“完全忽视”某个位置。**

掩码操作，就是为了强制实现这种“绝对的忽视”。通过给目标位置加上一个巨大的负数（-∞），使得 $e^{-\infty} \to 0$，这样经过 Softmax 后，该位置的权重被精确地归零。这些位置就不再是“占比很小的噪声”，而是被彻底从计算图中剪除。

计算案例：

假设我们有一个简单的注意力机制，其中查询、键和值都是相同的向量。我们的输入序列是 `[1, 2, 3, 0, 0]`，其中 `0` 表示填充符（如 `<PAD>`）。我们希望屏蔽掉这些填充符对应的位置。

（1）计算注意力分数

假设我们的注意力分数矩阵 S 是：

$$S = \begin{bmatrix} 1 & 2 & 3 & 0 & 0 \\ 2 & 4 & 6 & 0 & 0 \\ 3 & 6 & 9 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

在这里，第 4 行和第 5 行对应填充符，所以它们的注意力分数被设置为0。

（2）应用注意力掩码

我们的注意力掩码 M 将是一个5x5的矩阵，其中与填充符对应的位置（即第4行和第5行，以及第4列和第5列）被设置为负无穷，其余位置被设置为0。假设负无穷用一个非常大的负数（如-10000）来表示，那么 M 可能是：

$$M = \begin{bmatrix} 0 & 0 & 0 & -10000 & -10000 \\ 0 & 0 & 0 & -10000 & -10000 \\ 0 & 0 & 0 & -10000 & -10000 \\ -10000 & -10000 & -10000 & -10000 & -10000 \\ -10000 & -10000 & -10000 & -10000 & -10000 \end{bmatrix}$$

（3）计算Softmax前的注意力分数

将 S 和 M 相加，得到 S'：

$$S' = S + M = \begin{bmatrix} 1 & 2 & 3 & -10000 & -10000 \\ 2 & 4 & 6 & -10000 & -10000 \\ 3 & 6 & 9 & -10000 & -10000 \\ -10000 & -10000 & -10000 & -10000 & -10000 \\ -10000 & -10000 & -10000 & -10000 & -10000 \end{bmatrix}$$

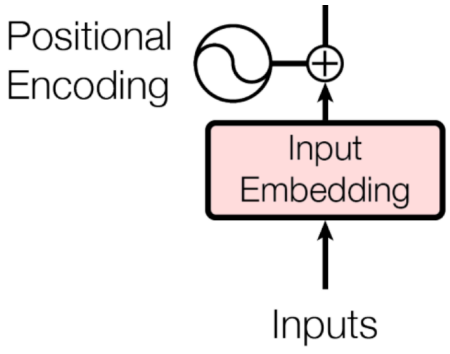
（4）计算Softmax

对 S' 的每一行进行Softmax操作。由于负无穷的存在，与填充符对应的位置的注意力权重将趋近于0。因此，最终的注意力权重矩阵 A 将只关注有效的位置。

（5）掩码的本质

注意力掩码的数学本质确实是：**通过加法介入，利用指数函数的极限性质，实现权重的绝对清零，从而防止无效信息的干扰。**

1.3 Encoder模块：Embedding + 位置编码



在进入 Self-Attention 之前，模型首先需要将离散的 token 转换为可以计算的向量形式。

1.3.1 为什么引入位置编码

仅有 Embedding 还不够，因为 Transformer 中的 Attention 机制本身有一个关键特点：

> **它不具备天然的序列顺序建模能力。**

也就是说，在 Attention 眼中，输入更像是一个“集合（set）”，而不是一个“有顺序的序列”。

（1）从 RNN 的角度理解这个问题

我们可以回顾一下之前学过的 RNN：

在 RNN 中，序列是**按时间步依次输入的**：

```
t1 → t2 → t3 → t4
```

每一步的计算都依赖前一步的隐藏状态：

$$h_t = f(h_{t-1}, x_t)$$

因此，RNN 天然就“带有顺序信息”，因为顺序被编码进了计算过程。

（2）Transformer 的不同之处

它的 Attention 是**一次性处理整个序列**，所有 token 同时进入模型，每个 token 之间的关系是通过计算得到的，而不是按顺序递归产生的。因此，如果不额外提供位置信息，模型看到的只是：

```
{我，爱，你}
```

而不是：

```
我 → 爱 → 你
```

（3）核心差异总结

模型	**顺序信息来源**
RNN	由“时间递归结构”天然提供
Transformer	需要“显式加入”

1.3.2 完整流程

（1）Token Embedding（词向量表示）

首先，每个 token（词或子词）都会通过一个“查表操作”映射为一个固定维度的向量。

可以简单理解为：token → 向量表示

例如：`“机器学习” → [0.12, -0.33, 0.87, ...]`

这个向量用于表达词的语义信息。

（2）位置编码（Positional Encoding）

因此，Transformer 会为每个 token 添加一个“位置信息向量”

```
最终输入 = Token Embedding + Position Encoding
```

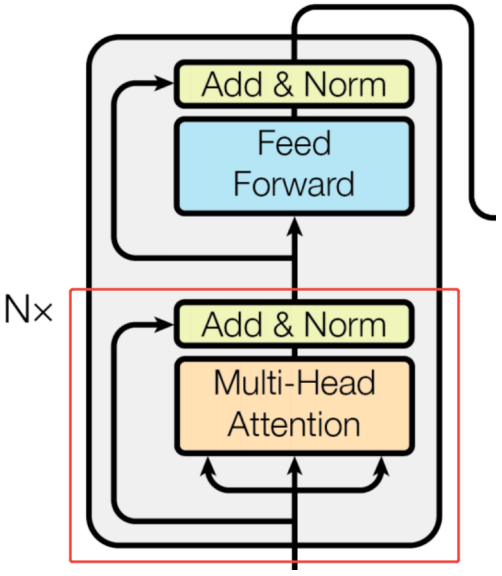
（3）处理结果

经过这一步之后：

- 每个 token 不仅有“语义信息”
- 还包含“位置信息”

因此模型输入不再只是词，而是：带有语义 + 顺序信息的向量序列

1.4 Encoder模块：Attention Sublayer（注意力子层）



1.4.1 Multi-Head Attention

多头注意力（Multi-Head Attention）机制，就像是一个“专家团队”在协同工作，每个“专家”都从不同的角度去理解问题。在Transformer模型中，我们使用了8个这样的“专家”，也就是8组不同的Q（Query）、K（Key）、V（Value）矩阵，这就是“多头”的含义。

处理流程：

（1）线性变换

首先，我们会对输入进行线性变换，生成多组Q、K、V矩阵。这就像是每个“专家”都在根据自己的专长，对输入进行不同的解读。

（2）并行计算

接下来，每组Q、K、V矩阵都会独立地进行缩放点积注意力计算。这就像是每个“专家”都在根据自己的解读，去关注输入中的不同部分。



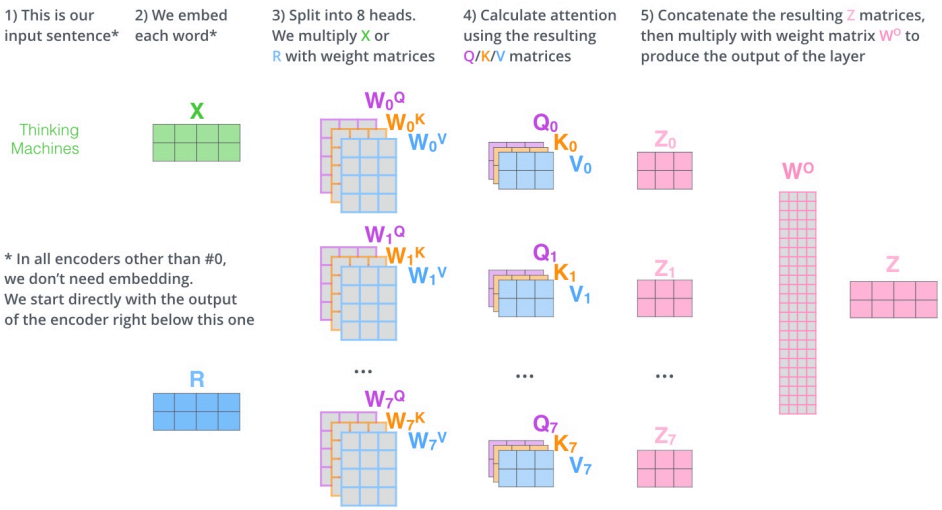
（3）合并结果

计算完所有组的注意力后，我们会将结果拼接在一起。这就像是所有“专家”都给出了自己的意见，然后我们综合这些意见，形成一个更全面的结论。

（4）最终变换

最后，我们会再经过一个线性变换，将拼接后的结果映射到最终的输出空间。这就像是综合了所有“专家”的意见后，我们做出了一个最终的决策。

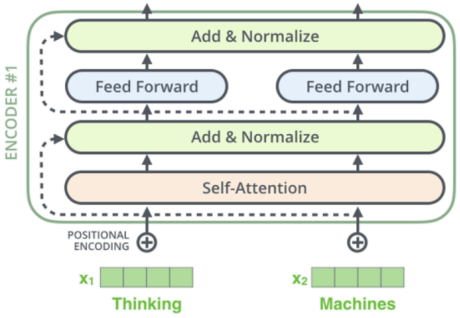
实际计算时，采用了矩阵合并的方式，一次性计算出最终的结果。



在Transformer架构中，图中的“R”通常指的是输入序列的表示矩阵（Representation matrix）。

具体来说，在Transformer的编码器部分，输入序列首先通过嵌入层（Embedding layer）转化为词嵌入（Word embeddings），这些词嵌入随后与位置编码（Positional encodings）相加，形成最终的输入表示矩阵“R”。

1.4.2 残差连接与层归一化（Add & Norm）



在多头注意力（Multi-Head Attention）计算完成后，Transformer 并不会直接将结果传入下一层，而是引入了一个非常关键的结构：**残差连接（Residual Connection）+ 层归一化（Layer Normalization）**，统称为 **Add & Norm**。

其核心计算过程如下：

$$\text{Output} = \text{LayerNorm}(X + \text{MHA}(X))$$

其中：

- X：输入表示（上一层输出或Embedding + Positional Encoding）
- $\text{MHA}(X)$：Multi-Head Attention，多头注意力计算
- $X + \text{MHA}(X)$：残差连接（Add）
- $\text{LayerNorm}(\cdot)$：层归一化（Norm）

1.4.3 LayerNorm

（1）输入形状

我们可以用一个三维张量来理解 NLP 输入：

$$(batch,\ token,\ hidden_dim)$$

其中：

- batch：批次大小
- token：序列长度
- hidden_dim：每个 token 的向量表示

（2）LayerNorm的效果

在 Transformer 中，每一个 token 都对应一个 hidden_dim 维的向量，相当于把每一个Token转换成词向量。LayerNorm 的作用就是：

> **对每一个 token 的向量内部进行归一化**

也就是说：

- 不看 batch 中其他样本
- 不看同一序列中的其他 token
- 只处理“当前 token 自己的向量”

（3）具体例子

✔ Step 1：计算均值

$$\mu = \frac{2+0+2+4+6}{5} = \frac{14}{5} = 2.8$$

✔ Step 2：计算方差

$$\sigma^2 = \frac{1}{5}\sum(x_i - \mu)^2$$

逐项计算：

(2 - 2.8)² = 0.64

(0 - 2.8)² = 7.84

(2 - 2.8)² = 0.64

(4 - 2.8)² = 1.44

(6 - 2.8)² = 10.24

$$\sigma^2 = \frac{0.64+7.84+0.64+1.44+10.24}{5} = \frac{20.8}{5} = 4.16$$

✔ Step 3：计算标准差

$$\sigma = \sqrt{4.16} \approx 2.04$$

✔ Step 4：标准化（LayerNorm核心步骤）

公式：

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

这里忽略 ε（极小值），直接计算：

分别计算每一维：

- $\hat{x}_1 = \frac{2-2.8}{2.04} = \frac{-0.8}{2.04} \approx -0.39$
- $\hat{x}_2 = \frac{0-2.8}{2.04} = \frac{-2.8}{2.04} \approx -1.37$
- $\hat{x}_3 = \frac{2-2.8}{2.04} \approx -0.39$
- $\hat{x}_4 = \frac{4-2.8}{2.04} = \frac{1.2}{2.04} \approx 0.59$
- $\hat{x}_5 = \frac{6-2.8}{2.04} = \frac{3.2}{2.04} \approx 1.57$

✔ Step 5：最终结果

```
LayerNorm(token) ≈ [-0.39, -1.37, -0.39, 0.59, 1.57]
```

（4）意义

LayerNorm 的计算本质不是“数学操作”，而是在做一件非常具体的事情：

> **把每个 token 的向量，转换到一个“统一尺度的表达空间”中。**

✔ 1. 从本质问题开始

在神经网络中，每一层输出的向量都有一个问题：

> 不同 token / 不同样本的“数值尺度”可能完全不一样

比如：

- 有的 token 向量整体很大（如 10, 20, 30）
- 有的 token 向量整体很小（如 0.1, 0.2, 0.3）

这会导致一个问题，后面的网络不知道该“信谁更多”

✔ 2. LayerNorm 在解决什么问题？

LayerNorm 的目标是：**消除“数值尺度差异”，只保留“方向和相对结构”**

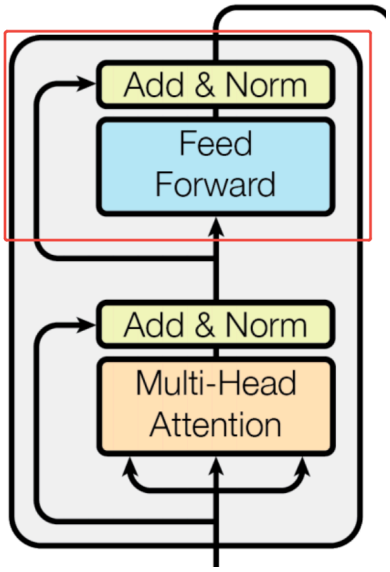
换句话说，我们不关心这个 token 的绝对大小，而关心：

- 哪些维度更重要

- 向量内部的结构关系
- 表达的“语义形状”

✔ 3. 一句话本质理解：**“把每个 token 的向量标准化成一个可比较的语义空间”**

1.5 Encoder模块：Feed Forward Sublayer（前馈网络子层）



1.5.1 FFN（前馈神经网络子层）

在 Transformer 中，FFN（Feed Forward Network，前馈神经网络）是 Encoder 中的第二个核心子层。

它的结构非常简单，通常由两层全连接网络组成：

> 线性变换 → 激活函数 → 线性变换

（1）基本结构

FFN 的计算过程如下：

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2$$

语言的语义本质上是高维复杂流形（manifold）。文本表示是将离散、高复杂度符号序列投影到连续向量空间 $\mathbb{R}^d$ 的过程。维度 d 决定了这个投影的“带宽”或自由度（degrees of freedom）。

低 d 相当于强有损压缩，失真大；高 d 允许更忠实地近似原始语义流形。

其中：

- 第一层：升维，可以增加表达能力
- 第二层：降维，恢复原始维度

（2）作用理解与机制本质

FFN（Feed Forward Network）的作用是：

> **对每个 token 的表示进行非线性特征变换与增强**

在 Transformer 中：

- Attention 负责建模 token 之间的关系（信息交互）
- FFN 负责对每个 token 的表示进行独立的特征映射（非线性变换）

（3）总结

FFN 是 Transformer 中非常基础但重要的组成部分，它与 Multi-Head Attention 一起构成 Encoder 的两个核心子层，共同完成信息建模与特征变换。

1.5.2 ADD & Norm

FFN 计算完成后，Transformer 并不会直接将结果传入下一层，而是引入了一个非常关键的结构：**残差连接（Residual Connection）+ 层归一化（Layer Normalization）**，统称为 **Add & Norm**。

其核心计算过程如下：

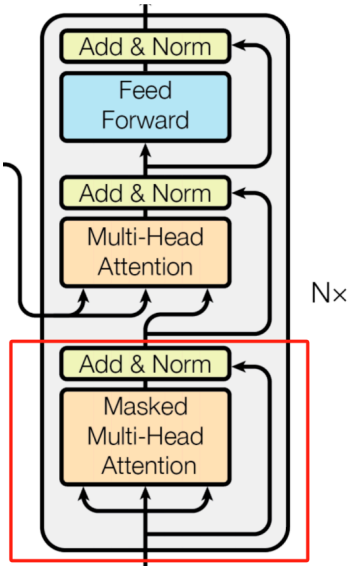
$$\text{Output} = \text{LayerNorm}(X + \text{FFN}(X))$$

这里，我们可以把两个子层进行统一表述：

$$\text{Output} = \text{LayerNorm}(X + \text{SubLayer}(X))$$

这里的SubLayer，既可以指代 MHA，也可以指代 FFN。

1.6 Decoder模块：Masked Attention Sublayer（注意力子层）



1.6.1 Masked Multi-Head Attention

（1）Mask遮罩

Decoder 的第一个注意力子层，其内部结构与 Encoder 的 Multi-Head Attention 完全一致：同样是多组 Q/K/V 的线性变换，同样是缩放点积注意力计算，同样有残差连接和层归一化。

唯一的区别，也是这个子层的**灵魂**，就在于多了一个 **Mask（掩码）操作**。它被插入在注意力分数计算之后、SoftMax 归一化之前。

Mask 的作用可以一句话概括：强制让当前位置的 token 永远看不到未来的信息。

（2）遮罩原因

Decoder 的训练目标是"给定前文，预测下一个词"。

在训练时，为了并行效率，我们会一次性把整个目标句子喂给模型。如果不加掩码，模型在预测第 t 个词时，就能直接"偷看"到第 $t+1$ 个词甚至更后面的正确答案。这就好比做听力填空，却先把全文给了你——模型根本学不会从残缺上下文去推理，只会简单地抄答案。

因此，Mask 的加入维护了**自回归生成**的因果逻辑：在预测时刻 t 时，模型只能依赖时刻 $0, 1, \ldots, t$ 的输入，不能依赖时刻 $t+1$ 及之后的部分。正是这个"不完整的信息视角"，逼迫模型真正学会根据前缀去推演未来。

所以，Masked Multi-Head Attention 本质上就是：

> **标准 MHA + SoftMax 前遮住未来位置**

1.6.2 Causal Mask（因果掩码）

（1）与注意力掩码的异同

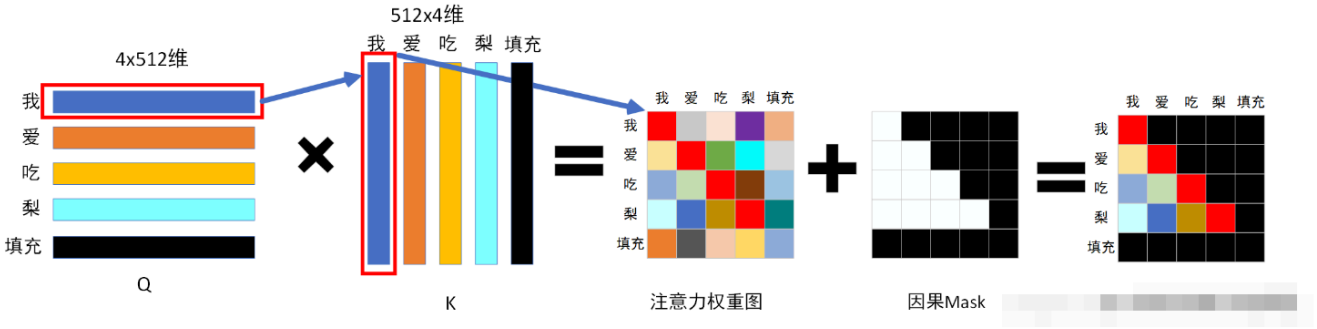
相同点：操作手法完全一致

无论是 Encoder 的填充掩码，还是 Decoder 的因果掩码，它们在代码实现层面都是同一个动作：**构造一个由 00 和 $-\infty$ 组成的矩阵，然后加到注意力分数上**。从这个角度看，它们都叫"掩码"是完全合理的。

不同点：屏蔽的对象和逻辑源头

- **填充掩码**：屏蔽的是**无意义的输入内容**（`[PAD]`）。这是一种**数据侧**的处理，目的是不让人为补长操作产生噪声，掩码形态随输入不同而动态变化。
- **因果掩码**：屏蔽的是**未来的时间步**。这是一种**模型结构侧**的强制约束，目的是维护自回归生成的因果逻辑，掩码形态是一个固定不变的上三角矩阵。

（2）具体实现



因果掩码是一个方阵，尺寸为序列长度 × 序列长度。其规则非常简单：

- 对于位置 i 和 j，若 $i \geq j$（即当前位置及之前的位置），掩码值为 0，表示**可见**；
- 若 $i < j$（即未来的位置），掩码值为 $-\infty$（实践中用一个极大的负数），表示**不可见。**

以长度为 4 的序列为例，因果掩码矩阵长这样（行表示要预测的位置，列表示能看到的范围）：

```
      t0   t1   t2   t3
t0 [   0, -∞,  -∞,  -∞ ]
t1 [   0,  0,  -∞,  -∞ ]
t2 [   0,  0,   0,  -∞ ]
t3 [   0,  0,   0,   0 ]
```

在计算注意力时，这个矩阵会被加到注意力分数 QK^T 上：

$$\text{Masked Scores} = QK^T + \text{Causal Mask}$$

（3）示例

假设 Decoder 输入序列长度为 3，即 $X = [x_1, x_2, x_3]$。下面用具体数字走完整个因果掩码的矩阵计算流程。

Step 1：原始注意力分数矩阵

假设三个位置的查询 Q 和键 K 已经算完，点积缩放后得到一个 3×3 的分数矩阵（行表示查询位置，列表示键位置）：

$$\text{Scores} = \begin{bmatrix} 2.0 & 1.0 & 0.5 \\ 1.5 & 3.0 & 2.0 \\ 0.8 & 2.2 & 4.0 \end{bmatrix}$$

Step 2：因果掩码矩阵

序列长度 $n = 3$ 的因果掩码矩阵是一个上三角为 $-\infty$ 的矩阵：

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

Step 3：将掩码加到分数矩阵上

$$\text{Scores}_{\text{masked}} = \text{Scores} + M = \begin{bmatrix} 2.0 & -\infty & -\infty \\ 1.5 & 3.0 & -\infty \\ 0.8 & 2.2 & 4.0 \end{bmatrix}$$

第1行只能看到位置1（$-\infty$ 把位置2、3屏蔽）；

第2行能看到位置1、2，看不到位置3；

第3行能看到全部位置。

Step 4：对每一行做 Softmax

Softmax 时，$e^{-\infty}$ 趋于 0，对应权重变为 0。结果注意力权重矩阵为：

$$\text{Weights} = \begin{bmatrix} 1.00 & 0 & 0 \\ 0.18 & 0.82 & 0 \\ 0.03 & 0.12 & 0.85 \end{bmatrix}$$

Step 5：加权求和得到输出

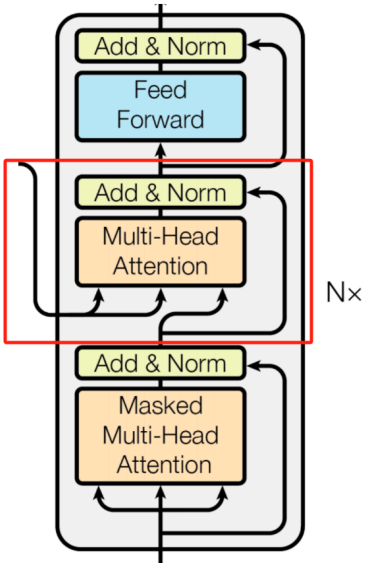
用这个权重矩阵乘以值矩阵 V（假设 $V = [v_1, v_2, v_3]^T$），即得到每个位置的输出：

- $\text{Output}_1 = 1.00 \cdot v_1$
- $\text{Output}_2 = 0.18 \cdot v_1 + 0.82 \cdot v_2$
- $\text{Output}_3 = 0.03 \cdot v_1 + 0.12 \cdot v_2 + 0.85 \cdot v_3$

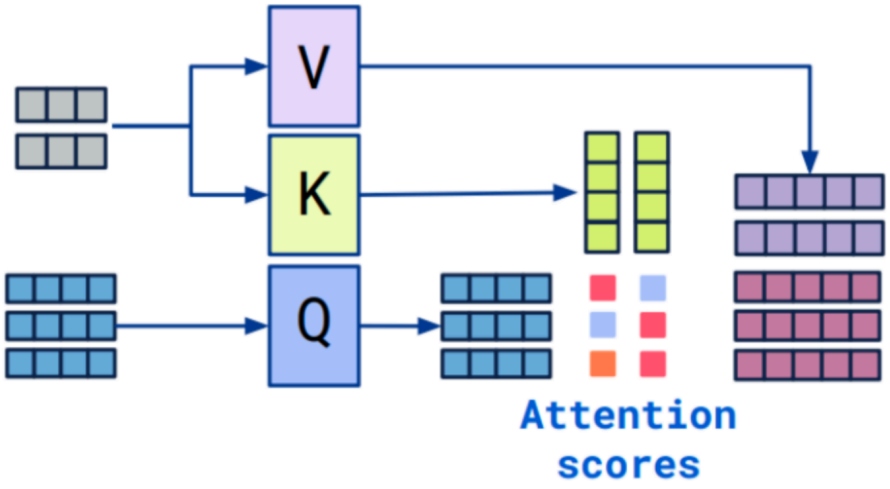
这样，每个位置都严格只看当前及过去的 token，未来信息被完全阻断。

那些被设成 $-\infty$ 的位置，经过 SoftMax 后会变成趋近于 0 的权重，相当于被彻底抹除。模型最终对 V 进行加权求和时，便完全不会触及任何未来 token 的信息，从而严格保证了“只看过去，不看未来”的因果约束。

1.7 Decoder模块：Cross-Attention Sublayer（交叉注意力子层）



1.7.1 Cross-Attention



在标准 Transformer 中，Cross-Attention 特指**解码器（Decoder）中的第二个多头注意力子层**（介于“掩码自注意力”和“前馈网络”之间），Cross-Attention 机制扮演着连接 Encoder 和 Decoder 的关键角色。

它使得 Decoder 中的每个位置能够获取到 Encoder 所有位置的信息，从而实现 Encoder-Decoder 之间的信息交互。

其 **Q、K、V 的来源绝对分离**：

- **Q（查询）**：来自**解码器**的**上一子层输出**（即掩码自注意力层的输出，代表“已生成的目标序列”）。
- **K（键）** 和 **V（值）**：共同来自 **编码器（Encoder）的最终输出**（代表“完整的源输入序列”）。

数学表达为：

$$Cross_Attention = Attention(Q_{Decoder}, K_{Encoder}, V_{Encoder})$$

1.7.2 核心原因

（1）完成“序列到序列（Seq2Seq）”的对齐任务（最根本原因）

Transformer 最初是为机器翻译设计的。

源语言（如英语）和目标语言（如中文）的单词顺序、语法结构不对等。Self-Attention 无法处理两个不同长度序列间的映射关系。Cross-Attention 让解码器的**每一个输出位置**，都能动态地给编码器的**所有输入位置**分配注意力权重，自动实现“软对齐”（Soft Alignment）。例如翻译“I love you”时，生成“我”时，Q 会重点查询编码器中“I”对应的 K/V。

（2）解耦“理解”与“生成”的职责

编码器只负责理解源句：它把所有语义压缩成一组固定的 K/V 表征，不再改动。

解码器负责自回归生成：每生成一个新词，Q 都在变化。Cross-Attention 的作用是**让动态的 Q 去静态的 K/V 库中检索。**

如果不设 Cross-Attention，而强行把源句和目标句拼一起做 Self-Attention，会让编码器在理解时受未来目标词干扰（破坏双向理解），解码器在生成时又受源句的全局混杂信息干扰。Cross-Attention 将这两步**物理隔离**，保证了训练的稳定性和表征的纯净性。

（3）充当解码器获取“外部上下文”的唯一通道

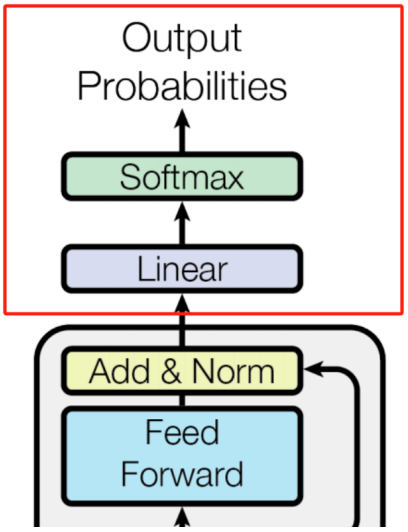
掩码自注意力只允许看到“当前已生成的前缀词”，它完全不知道源句长什么样。

如果没有 Cross-Attention，解码器就成了一个脱离输入文本的“胡言乱语”生成器。Cross-Attention 是**源句信息注入解码器的唯一入口**，让生成结果有据可依（条件生成）。

总结一句话（标准 Transformer 语境下）：

Cross-Attention = **解码器利用编码器表征进行动态查询。**它存在的唯一技术动机是 **将源序列的全局语义动态映射到目标序列的每个生成步长中**，从而打破单序列注意力的封闭性，实现跨序列的精准信息交互与对齐。

1.8 Decoder模块：解码器输出与生成机制



经过堆叠的 N 个解码器层（每层均包含掩码自注意力和交叉注意力）的层层抽象后，原始输入序列（源语言）的信息与已生成的目标序列前缀信息，已被融合进一个高维的特征向量空间中。

此时，解码器顶层的输出并不是我们直接能读懂的单词，而是一个维度为 d_{model} 的**浮点数特征向量**。接下来的任务，就是如何将这个“机器内部表示”翻译回人类可读的离散词汇。

1.8.1 线性投影层

顶层解码器输出的向量维度是固定的 d_{model}（如 512 或 768），而我们需要预测的目标词汇表的大小为 $|V|$（如 3 万或 10 万）。由于两者维度不一致，第一步需要通过一个全连接线性层（不带偏置或带偏置）进行维度变换：

$$\text{logits} = \text{Linear}(h) = h \cdot W^T + b$$

- 其中 $h \in \mathbb{R}^{d_{model}}$ 是顶层解码器某个位置输出的特征向量。
- W 是权重矩阵，尺寸为 $|V| \times d_{model}$。
- 经过这一步，原本的 d_{model} 维向量被映射为一个 $|V|$ 维的**原始得分向量（logits）**。logits 中的每一个数值代表了该位置对应词汇的“绝对匹配程度”，数值越高，表示模型认为该词与当前上下文越契合。注意，此时的 logits 取值是未经过归一化的实数（可正可负）。

1.8.2 Softmax 归一化

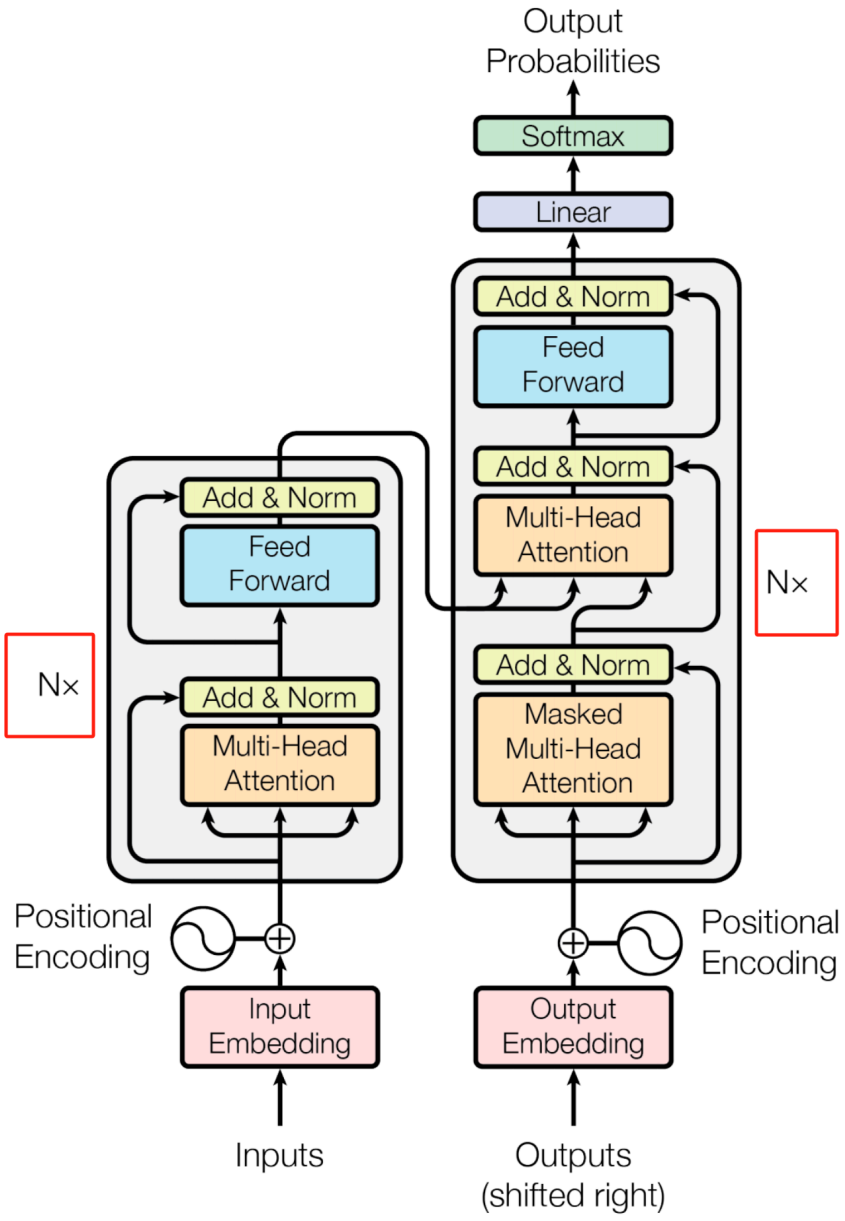
仅仅有得分是不够的，因为不同样本间的得分量纲不统一，且我们需要一个明确的指标来决定选哪个词。因此，需要将 logits 输入 Softmax 激活函数，将其转化为一个**有效的概率分布**：

$$P(w_i|\text{context}) = \frac{e^{\text{logits}_i}}{\sum_{j=1}^{|V|} e^{\text{logits}_j}}$$

Softmax 操作具有两个显著特性，使其成为输出的必备环节：

- **归一性**：所有词汇的概率之和严格等于 1，这使得输出可以被解释为模型在当前上下文下的“置信度分布”。
- **相对放大效应**：指数函数 e^x 会显著拉大 logits 中高分与低分之间的差距，让概率集中到最可能的少数词汇上，从而降低低概率词汇被误选的风险。

1.9 Transformer 宏观视角与范式意义



1.9.1 从原始文本到输出概率（宏观流程图）

1.9.2 为什么废弃 RNN

1.9.3 Transformer的优越性

在WMT 2014英语-德语翻译任务中，大型transformer模型（Transformer (big)）比以前报道的最佳模型（包括整合模型）高出2.0个BLEU以上，确立了一个全新的最高BLEU分数为28.4。 训练在8 个P100 GPU上花费3.5 天。

在WMT 2014英语-法语翻译任务中，大型transformer模型的BLEU得分为41.0，超过了之前发布的所有单一模型，训练成本低于先前最先进模型的1／4 。

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

2.BERT：秦王扫六合

在前面的小节中，我们已经学习了 Transformer。

Transformer 解决的是序列建模结构的问题：它通过 Self-Attention，让模型不再依赖 RNN 的逐步递归，而是可以让序列中的每个 Token 直接和其他 Token 建立联系。

从 NLP 的发展历程来看，真正贯穿始终的主线，是文本表示方式的不断升级，分词技术在变，算法模型在变，但它们都服务于同一个目标：

> **把人类语言转化成机器可以理解、可以计算、可以学习的表示。**

因此，理解 BERT 的历史地位，不能只看它用了 Transformer Encoder，也不能只看它在很多 NLU 任务上效果很好。更关键的是：

> **BERT 第一次把“文本表示学习”和“下游任务模型”高度统一到同一个预训练模型底座中。**

这才是 BERT 真正“秦王扫六合”的地方。

2.1 BERT 的背景

在 BERT 出现之前，NLP 虽然经历了规则时代、统计学习时代和深度学习时代，但每个时代都存在一个共同问题：

> **文本表示与任务模型之间始终是割裂的。**

2.1.1 规则时代：文本表示依赖人工规则

最早的 NLP 系统，主要依赖人工规则。

- 做中文分词，要写词典和匹配规则。
- 做情感分析，要整理情感词典。
- 做信息抽取，要设计正则表达式和模板。

这个时期，机器并没有真正学习文本表示。所谓“表示”，本质上来自人类提前写好的规则。

例如：

```
如果句子中出现“非常好”“喜欢”“满意”
就认为它偏正向情感

如果句子中出现“差评”“垃圾”“失望”
就认为它偏负向情感
```

这时的文本表示不是模型学出来的，而是人类设计出来的。所以规则时代的核心特征是：

> **人工规则就是文本表示，规则系统就是任务模型。**

- 规则覆盖到了，系统就能处理；
- 规则没有覆盖到，系统就会失效。

这个阶段最大的问题是：系统的能力边界完全依赖人工经验。换句话说，机器不是在理解语言，而是在执行人类写好的语言规则。

2.1.2 统计学习时代：从人工规则到特征工程 + 算法模型

后来，NLP 进入统计学习时代。相比规则时代，统计学习时代最大的变化是，模型不再完全依赖人手写判断规则，而是开始从数据中学习规律。但是，这个阶段并没有直接让模型自己理解文本。

它采用的是另一种方式：

> **先由人设计文本特征，再把特征交给传统机器学习模型。**

也就是说，统计学习时代的典型范式是：

```
原始文本
    ↓
分词
    ↓
人工特征工程
    ↓
传统机器学习模型
    ↓
任务输出
```

- 做文本分类时，我们可以先把文本转成 TF-IDF 向量，然后交给 SVM、朴素贝叶斯、逻辑回归等模型。
- 做序列标注时，可以人工设计词性、上下文窗口、词形、位置等特征，然后交给 HMM、CRF 等模型。
- 做信息检索时，可以通过倒排索引、TF-IDF、BM25 等方式表示文本和查询之间的相关性。

这个时代相比规则时代已经进步很多，规则时代是：

```
人直接写判断逻辑
```

统计学习时代变成了：

```
人设计文本特征
模型学习特征与结果之间的关系
```

但是问题仍然存在，因为文本表示依然主要依赖人工特征工程。

- 你要做分类，就设计分类特征。
- 你要做序列标注，就设计序列标注特征。
- 你要做信息检索，就设计检索特征。
- 你要做机器翻译，又需要另一套表示和模型体系。

这就导致一个很严重的问题：

> **文本表示和算法模型是割裂的，不同任务之间也是割裂的。**

文本表示靠人设计，算法模型负责学习，表示是一部分，模型是一部分。任务换了，特征往往也要重新设计，模型也可能要重新选择。所以统计学习时代虽然比规则时代更强，但本质上仍然是“诸侯割据”。

- 每个任务都有自己的特征工程。
- 每个任务都有自己的算法模型。
- 每个系统都像一座单独修建的城池。

2.1.3 深度学习时代：从人工特征工程到模型自动学习表示

进入深度学习时代之后，NLP 发生了一次关键变化：

> **文本表示不再主要依赖人工特征工程，而是开始由模型自动学习。**

Word2Vec 的出现，让词不再只是离散符号，而是变成了连续向量。

例如：

```
“国王” → 一个向量
“王后” → 一个向量
“北京” → 一个向量
“上海” → 一个向量
```

这些向量不是人工手写的规则，也不是人工设计的 TF-IDF 特征，而是模型从大规模语料中学习出来的。这意味着，NLP 开始从“人工设计表示”进入“模型学习表示”的阶段。

随后，TextCNN、RNN、LSTM、BiLSTM-CRF、Seq2Seq、Attention 等模型不断出现。

它们进一步增强了模型对文本的表示能力。

- RNN 可以按照顺序处理文本。
- LSTM 可以缓解长距离依赖问题。
- Seq2Seq 可以把输入序列编码成向量，再生成目标序列。
- Attention 可以让模型在处理当前信息时关注更关键的位置。
- Transformer 则通过 Self-Attention 进一步提升了全局建模能力。

到了这个阶段，NLP 的能力确实大幅提升，因为模型终于可以自己学习表示了。但是，深度学习时代仍然没有彻底完成统一，原因在于：

> **不同任务仍然需要不同的模型结构。**

- 文本分类可能用 TextCNN、TextRNN。
- 机器翻译可能用 Seq2Seq + Attention。
- 阅读理解又需要专门设计输入交互和答案预测结构。

也就是说，深度学习解决了“人工特征工程”的问题，但没有彻底解决“任务模型割裂”的问题。

统计学习时代是：

```
人工设计特征 + 不同算法模型
```

深度学习时代变成了：

```
模型自动学习表示 + 不同任务模型
```

这是一次巨大进步，但还不是最终统一，因为每个任务仍然有自己的模型打法。神经网络让各路诸侯都变强了，但诸侯仍然是诸侯。

2.1.4 Transformer：强大的结构基础，但还不是统一范式

Transformer 的出现，让序列建模能力进一步提升。它通过 Self-Attention，让每个 Token 都可以直接看到整个序列中的其他 Token，从而更好地建模全局上下文关系。这一步非常重要，因为它提供了一种强大的文本编码结构。

但是，在 Transformer 刚出现时，它本身主要还是作为一种模型架构存在，最典型的应用是机器翻译中的 Encoder-Decoder 结构。

也就是说，Transformer 解决的是：

> **用什么结构更好地建模序列？**

但它还没有完全回答：

> **能不能训练一个通用语言底座，让大量 NLU 任务共用同一个文本表示模型？**

这个问题，正是 BERT 要解决的。

2.1.5 BERT：统一文本表示与任务模型

BERT 的关键意义就在这里，它不是简单地把 Transformer Encoder 拿来用了一下，而是把 Transformer Encoder 变成了一个可以被大量 NLU 任务共享的预训练语言模型底座。

BERT 的范式是：

```
大规模语料预训练
    ↓
学习通用上下文文本表示
    ↓
下游任务微调
    ↓
适配分类、匹配、标注、问答等任务
```

这一步真正改变了 NLP 的组织方式。

在 BERT 之前，常见范式是：

```
任务 A → 设计表示 A → 选择模型 A
任务 B → 设计表示 B → 选择模型 B
任务 C → 设计表示 C → 选择模型 C
```

BERT 之后，范式变成：

```
任务 A → BERT 表示 + 任务头 A
任务 B → BERT 表示 + 任务头 B
任务 C → BERT 表示 + 任务头 C
```

也就是说，底层文本表示统一了。

- 分类任务用 BERT。
- 句子匹配用 BERT。
- 命名实体识别用 BERT。
- 阅读理解也用 BERT。

不同任务之间的差异，不再主要体现在底层模型结构上，而是体现在最后接什么任务头。

- 做文本分类，可以使用 `[CLS]` 向量接分类头。
- 做句子匹配，可以输入句子对，再接分类头。
- 做命名实体识别，可以对每个 Token 接标注头。
- 做阅读理解，可以预测答案的起始位置和结束位置。

所以 BERT 的统一不是一句空话。

它统一了三个层面：

```
第一，统一了文本表示：
不再为每个任务单独设计表示，而是使用 BERT 学到的上下文语义表示。

第二，统一了模型底座：
不再为每个 NLU 任务从零设计模型，而是共享 Transformer Encoder 预训练模型。

第三，统一了训练范式：
不再完全依赖任务数据从头训练，而是采用“预训练 + 微调”。
```

这就是 BERT 的历史地位。

2.1.6 为什么说 BERT 是“秦王扫六合”

所以，我们说 BERT 是“秦王扫六合”，重点不是说它解决了所有 NLP 任务，这个说法不准确。

BERT 主要横扫的是 NLU 任务领域，也就是文本理解任务。

这条线非常重要：

```
规则时代：
人工规则 = 文本表示 + 任务逻辑

统计学习时代：
人工特征工程 + 传统算法模型

深度学习时代：
模型自动学习表示 + 任务专用神经网络

BERT 时代：
预训练模型统一学习表示 + 简单任务头微调
```

这就是从“分裂”走向“统一”的过程，过去，NLP 像一个诸侯割据的战场。而 BERT 出现之后，大量 NLU 任务终于有了共同的底座。

这就是“秦王扫六合”的真正含义，它扫的不是所有 NLP 问题，而是扫掉了大量 NLU 任务长期以来“表示割裂、模型割裂、训练割裂”的局面。

一句话总结：

> **BERT 的出现，标志着 NLP 从“人工设计表示 + 任务专用模型”的割裂时代，进入了“预训练语言模型统一表示 + 任务头微调”的统一时代。**

2.2 从 Word2Vec 到 ELMo：从“存词向量”到“存模型参数”

在讲 BERT 架构之前，我们先要区分两种完全不同的文本表示方式：

```
第一种：训练好每个词的固定向量，使用时直接查表。

第二种：训练好一个上下文编码模型，使用时根据当前句子动态生成表示。
```

Word2Vec 属于第一种，ELMo 和 BERT 更接近第二种。

这个区别非常关键，因为它决定了模型到底能不能根据上下文生成不同的词表示。

2.2.1 Word2Vec：训练完成后，核心产物是词向量表

Word2Vec 在训练时确实会利用上下文，例如 Skip-Gram 的思想是：

```
用中心词预测周围词
```

CBOW 的思想是：

```
用周围词预测中心词
```

所以，Word2Vec 并不是完全不看上下文。但是问题在于，**上下文只参与训练过程，训练完成后，每个词最终仍然只有一个固定向量。**训练结束后，我们真正得到的是一张词向量表：

```
word2vec_embedding = {
    "苹果": [0.21, -0.15, 0.73, ...],
    "银行": [0.34,  0.08, -0.41, ...],
    "学习": [0.12,  0.56,  0.19, ...]
}
```

之后模型在使用时，看到“苹果”这个词，就直接从表里取出同一个向量。

例如：

```
我今天吃了一个苹果。
苹果发布了最新款手机。
```

这两个句子中的“苹果”含义不同：

```
第一个苹果：水果
第二个苹果：公司
```

但在 Word2Vec 中，它们查到的是同一个向量。所以 Word2Vec 的“静态性”不在于训练时没有上下文，而在于：

> **无论上下文怎么变，同一个词最终只能对应一个固定表示。**

这就像我们把一个词在各种语境中的含义全部压缩进一个向量里。一个多义词出现过很多不同语境，最终只能形成一个“混合语义向量”。它可能偏向高频语义，也可能混入少量其他语义，但无论如何，它无法在不同句子中临时切换表示。

所以 Word2Vec 解决了一个重要问题：

> **如何让词从离散符号变成连续向量。**

但它没有解决另一个问题：

> **同一个词在不同上下文中如何生成不同表示。**

2.2.2 ELMo：训练完成后，核心产物是上下文编码器参数

实际上，ELMo 真正重要的地方，并不是简单地“把 LSTM 换成 BiLSTM”。如果只是讨论结构，LSTM、BiLSTM、GRU、CNN、Attention 都只是不同的编码器选择。

真正让 ELMo 和 Word2Vec 拉开差距的，是它的训练与使用方式发生了变化。Word2Vec 训练完成后，主要得到的是一张词向量表，ELMo 训练完成后，核心产物不再是一张简单的“词向量表”，而是一套预训练好的语言模型参数。

具体来说，ELMo 使用的是双向 LSTM 语言模型，它会让句子分别经过：

```
前向 LSTM：从左到右读取句子
后向 LSTM：从右到左读取句子
```

然后根据当前词所在的句子环境，生成这个词的上下文表示。也就是说，ELMo 的使用方式不是简单查表：

```
苹果 → 固定向量
```

而是：

```
句子输入
   ↓
BiLSTM 编码
   ↓
生成每个词在当前句子中的表示
```

所以对于下面两个句子：

```
我今天吃了一个苹果。
苹果发布了最新款手机。
```

ELMo 会把整个句子输入 BiLSTM，然后，“苹果”的表示会受到周围词的影响。

- 在第一个句子中，“吃了一个”会让“苹果”的表示更偏向水果。
- 在第二个句子中，“发布”“手机”会让“苹果”的表示更偏向公司。

所以 ELMo 本质上已经和 Word2Vec 不同了。

Word2Vec 是：

```
训练阶段利用上下文
    ↓
最终保存每个词的固定向量
    ↓
使用时查表
```

ELMo 是：

```
训练阶段学习语言模型参数
    ↓
最终保存上下文编码器参数
    ↓
使用时根据当前句子生成表示
```

这个差别非常大，因此，我们可以说：

> **ELMo 已经从“静态词向量表”进入了“预训练上下文编码器”的阶段。**

这也是为什么说 ELMo 已经有 BERT 的味道。因为它们都不是简单保存每个词的固定向量，而是保存一个能够根据输入句子生成表示的模型。

2.2.3 ELMo 的局限：方向对了，但兵器落后了

既然 ELMo 已经这么先进，为什么后来没有成为主线，而是被 BERT 接管？原因不在于 ELMo 的思想错了，恰恰相反，ELMo 的方向非常正确。

它已经意识到：

> **词的表示不能只看词本身，而应该由上下文决定。**

但问题在于，ELMo 使用的技术底座还是 LSTM，这就像一把削铁如泥的宝刀。

- 在冷兵器时代，它当然是神兵利器。
- 但是当战场进入飞机、大炮、装甲集群的时代，再锋利的刀，也很难决定战争走向。

ELMo 的 LSTM 底座主要有几个限制。

- 第一，LSTM 依赖顺序计算，它必须按照时间步一步一步处理序列，前一个位置计算完，后一个位置才能继续。这会限制训练效率，也不利于大规模并行训练。
- 第二，长距离信息传递路径太长，如果句子里两个词距离很远，LSTM 需要经过很多时间步才能把信息传过去，虽然 BiLSTM 比普通 RNN 强很多，但它仍然没有摆脱顺序传递的结构限制。
- 第三，双向信息是分开建模后再融合，ELMo 通常是：

```
前向 LSTM：从左到右建模
后向 LSTM：从右到左建模
最后融合两边表示
```

这和 BERT 不一样，BERT 使用的是 Transformer 的 Encoder，可以在 Self-Attention 中让每个 Token 直接和整句话里的其他 Token 建立联系。

也就是说，ELMo 是：

```
左向信息 + 右向信息 → 融合表示
```

BERT 是：

```
每一层中，每个 Token 都可以和全句 Token 直接交互
```

这就不是同一个时代的武器了。

- ELMo 是 LSTM 时代上下文表示的高峰。
- BERT 是 Transformer 时代预训练表示的开端。

所以 ELMo 的尴尬在于：

> **它的训练范式已经接近新时代，但模型结构仍然属于旧时代。**

更关键的是，BERT 来得太快了。

ELMo 刚刚证明“上下文表示”这条路是对的，BERT 就直接用 Transformer Encoder 和 MLM 把这条路推到了新的高度。

于是历史叙事就变成了：

```
Word2Vec 开启词向量时代；
ELMo 证明上下文表示方向；
BERT 完成预训练表示统一。
```

所以 ELMo 不是没价值，也不是没名气，更准确地说：

> **ELMo 是一个很重要的过渡模型，但它的光辉很快被 BERT 掩盖了。**

它像黎明前的闪电，照亮了方向；但真正把天照亮的，是后面升起来的 BERT。

2.3 BERT 架构

2.3.1 BERT 的核心思想

BERT 的核心思想可以概括为一句话：

> **使用 Transformer Encoder 对输入文本进行深度双向编码，得到每个 Token 的上下文表示，再将这些表示迁移到不同下游 NLU 任务中。**

这里有三个关键词。

第一，**Encoder-Only**：BERT 只使用 Transformer 的 Encoder 部分，不使用 Decoder。

第二，**深度双向编码**：BERT 使用 Transformer Encoder，每一层 Self-Attention 中，每个 Token 都可以直接和整句话中的其他 Token 建立联系。

第三，**预训练 + 微调**：BERT先通过大规模语料预训练 Transformer Encoder，让模型学到通用文本表示；然后在下游任务中接一个简单任务头进行微调。

所以 BERT 的整体范式是：

```
大规模语料
    ↓
预训练 BERT
    ↓
得到通用文本表示能力
    ↓
接任务头微调
    ↓
完成分类、匹配、标注、问答等 NLU 任务
```

一句话总结：

> **BERT 的核心不是设计一个新的任务模型，而是训练一个强大的 Transformer Encoder 文本表示底座。下游任务不再重新造模型，而是在这个底座上接任务头。**

2.3.2 BERT 的整体结构

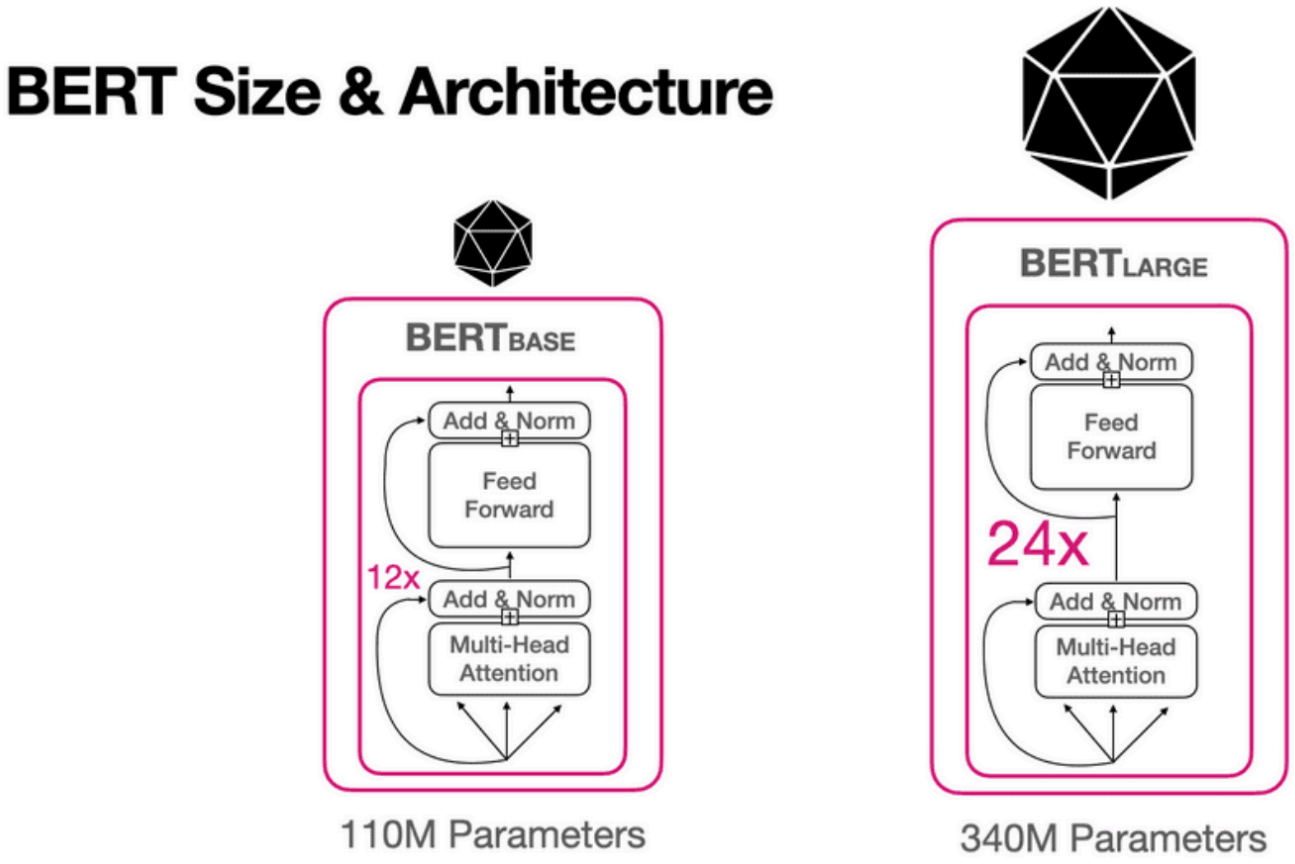
BERT 的整体结构非常直接：

```
输入文本 -> Embedding 层 -> Transformer Encoder × N -> 上下文表示 -> 任务头
```

其中最核心的部分是：

```
Transformer Encoder × N
```

也就是多层 Transformer Encoder 堆叠。



- BERT-Base 使用 12 层 Encoder。
- BERT-Large 使用 24 层 Encoder。

每一层 Encoder 内部结构与前面讲过的 Transformer Encoder 基本一致：

```
Multi-Head Self-Attention -> Add & Norm -> Feed Forward Network -> Add & Norm
```

2.3.3 BERT的输入与输出

（1）BERT 的输入格式

- BERT 的输入通常不是直接把一句原始文本扔进去，而是先经过分词器处理。

前面我们已经讲过 BPE、子词分词等方法，这里不再展开。需要注意的是，**原版 BERT 使用的是 WordPiece 分词**，它和 BPE 一样，都属于子词分词思想。

- 除此之外，BERT 还会加入特殊符号。

```
[CLS]：放在整个输入序列开头
[SEP]：用于分隔句子，或表示句子结束
```

例如单句输入：

```
[CLS] 我 喜欢 自然语言 处理 [SEP]
```

句子对输入：

```
[CLS] 句子A [SEP] 句子B [SEP]
```

（2）`[CLS]` 的重要性

因为 Transformer 的 Self-Attention 是全连接的，当文本经过多层 Encoder 后：

```
[CLS] -> 可以关注所有 Token

我 -> 可以关注所有 Token

喜欢 -> 可以关注所有 Token

...
```

所以经过很多层 Self-Attention 后：

```
[CLS]
```

这个位置会不断吸收整句话的信息，最终：

```
[CLS]输出向量 ≈ 整个句子的语义摘要
```

这里的 `[CLS]` 很重要，后面做文本分类时，通常会使用 `[CLS]` 对应的输出向量作为整句话的表示。

（3）BERT 的三种输入 Embedding

BERT 的每个位置输入，不只有 Token Embedding 和 Position Embedding，还有第三种 Embedding：

```
BERT 输入表示 = Token Embedding + Segment Embedding + Position Embedding
```

三者作用分别是：

```
Token Embedding：表示当前 Token 是什么。

Segment Embedding：表示当前 Token 属于句子 A 还是句子 B。

Position Embedding：表示当前 Token 在序列中的位置。
```

例如句子对任务：

```
[CLS] 今天天气很好 [SEP] 适合出去玩吗 [SEP]
```

其中，第一句话的 Token 属于 Segment A，第二句话的 Token 属于 Segment B。这样 BERT 才知道哪些 Token 来自第一句话，哪些 Token 来自第二句话。

（4）BERT 的输出是什么？

文本经过多层 Transformer Encoder 后，BERT 会为每个输入 Token 输出一个向量。也就是说，输入有多少个 Token，输出就有多少个上下文表示。

例如：

```
[CLS] 我 喜欢 自然 语言 处理 [SEP]
```

经过 BERT 后，会得到：

```
[CLS] → 向量
我    → 向量
喜欢  → 向量
自然  → 向量
语言  → 向量
处理  → 向量
[SEP] → 向量
```

这些向量不是静态词向量，而是经过 Transformer Encoder 融合上下文之后得到的表示，后面不同任务会使用不同位置的输出。

文本分类任务通常使用：

```
[CLS] 的输出向量
```

序列标注任务通常使用：

```
每个 Token 的输出向量
```

2.4 BERT的预训练

2.4.1 为什么需要预训练任务

前面我们已经知道，BERT 的本质是一个由多层 Transformer Encoder 堆叠而成的语言表示模型。当一段文本输入 BERT 后，BERT 会为每个 Token 输出一个上下文向量。

但是这里有一个关键问题，输出的这些向量一开始并不天然具备语言理解能力。也就是说，BERT 的参数在初始化时只是随机数。如果不经过训练，它输出的向量没有任何实际语义，不能真正表示“我”“喜欢”“自然语言处理”这些词在上下文中的含义。

因此，我们需要先通过某种训练任务，让 BERT 从大量文本中学习语言规律。

（1）BERT 不是直接为某个具体任务训练的

在传统深度学习模型中，我们通常会针对某个具体任务直接训练模型。**每换一个任务，就需要重新准备大量人工标注数据。**

而 BERT 的思路是先训练一个更通用的语言表示模型，BERT 的训练过程可以分成两个阶段：

```
预训练阶段：
在大规模通用文本上学习语言规律

微调阶段：
在具体任务数据上适配下游任务
```

可以理解为：

```
预训练：先让模型学会“读懂语言”
微调：再让模型学会“完成具体任务”
```

（2）预训练任务解决的核心问题

BERT 在预训练阶段要解决的问题是：

> 没有人工标注的情况下，如何让模型从大量文本中学习语言知识？

互联网上有海量文本，例如百科、新闻、书籍、网页文章等。这些文本本身没有情感标签、实体标签、分类标签，但是它们包含大量语言规律。

例如：

```
我喜欢自然语言处理。
```

这句话虽然没有人工标注标签，但它本身就包含很多可学习的信息：

```
“自然”“语言”“处理”经常组合在一起
“喜欢”后面通常接某种事物
“我”是动作或情感的主体
```

再比如：

```
小明打开书包，拿出了一本数学书。
```

模型可以从上下文中学习到：

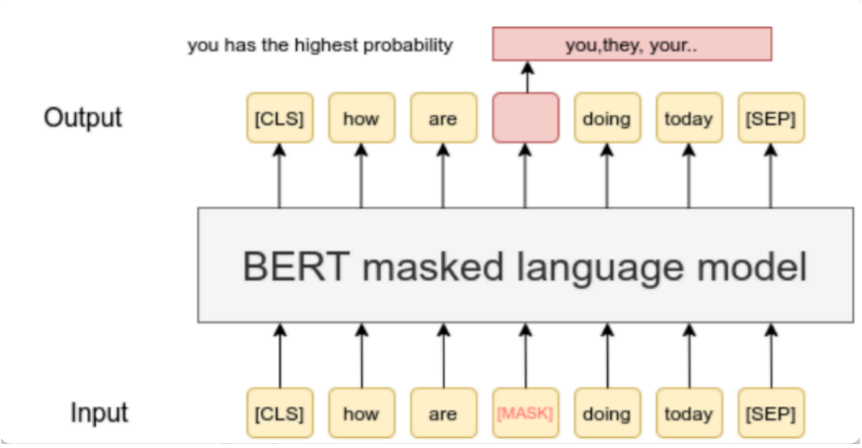
```
“书包”和“书”之间存在语义关联
“拿出”通常表示从某个容器中取出物体
“小明”是句子的主要人物
```

因此，BERT 的预训练任务，本质上就是利用文本自身构造训练信号，让模型在没有人工标注的情况下学习语言规律。这种训练方式通常称为：

> 自监督学习。

所谓自监督，就是不需要额外人工标注，而是从原始文本本身构造监督信号。

2.4.2 预训练任务1：MLM（Masked Language Model）



MLM 的全称是 **Masked Language Model**，通常翻译为**掩码语言模型。**它是 BERT 最核心的预训练任务之一，主要用来训练 BERT 理解 **Token 与上下文之间的关系。**

（1）MLM 要解决什么问题

MLM 的做法就是：

> 故意遮住句子中的一部分 Token，让模型根据上下文把它们预测回来。

例如：

```
原始句子：
我 喜欢 自然 语言 处理

遮蔽之后：
我 喜欢 [MASK] 语言 处理
```

模型需要根据上下文预测：

```
[MASK] = 自然
```

这就是 MLM 的基本思想。

（2）MLM Mask：替换 Token

BERT 的 MLM 里面，所谓“遮住一个词”，不是在注意力矩阵里加负无穷，而是把原来的 Token 替换成一个特殊 Token：

```
[MASK]
```

例如原句：

```
我 喜欢 自然 语言 处理
```

如果选中“自然”这个位置，输入会变成：

```
我 喜欢 [MASK] 语言 处理
```

这里的 `[MASK]` 本质上和 `[CLS]`、`[SEP]` 一样，是词表里的一个特殊 Token，它有自己的 Token ID，也有自己的 Embedding 向量。

所以模型真正看到的是：

```
我      → embedding
喜欢    → embedding
[MASK] → embedding
语言    → embedding
处理    → embedding
```

然后 `[MASK]` 这个位置仍然正常参与 Self-Attention，它不会被从注意力计算中删除。

（3）MLM 的“预测位置”通常用 label mask 控制

训练时还有一个东西，容易和前面的 mask 混在一起，MLM 不是对所有位置都计算损失，而是只对被选中的位置计算损失。

例如：

```
输入：
我 喜欢 [MASK] 语言 处理

原始句子：
我 喜欢 自然 语言 处理
```

训练目标可以理解为：

```
我      → 不预测
喜欢    → 不预测
[MASK] → 预测“自然”
语言    → 不预测
处理    → 不预测
```

在代码实现中，通常会用一个 `labels` 序列表示：

```
input_ids:
我   喜欢  [MASK]  语言  处理

labels:
-100 -100   自然    -100  -100
```

这里的 `-100` 不是负无穷，而是 PyTorch 里 `CrossEntropyLoss` 常用的忽略标记，意思是：

> 这个位置不参与损失计算。

只有被选中的预测位置，label 才是原始 Token ID。

所以最准确的说法是，MLM 里面有两层含义：

```
输入层面：
把被选中的 Token 替换成 [MASK] / 随机 Token / 原 Token

损失层面：
只在被选中的位置计算预测损失
```

不是：

```
在 Attention 矩阵里把被遮住的位置设为 -∞
```

（4）遮蔽比例：为什么通常选择 15%

在原始 BERT 中，训练时会随机选择输入序列中约 **15% 的 Token** 作为预测目标，这里选择 15% 是一个折中。

- 如果遮蔽比例太低，例如 5%，那么每个句子中需要预测的 Token 太少，模型从每个样本中获得的训练信号比较有限。
- 如果遮蔽比例太高，例如 50%，句子中大量信息被破坏，模型很难根据上下文进行有效判断。

例如：

```
我 [MASK] [MASK] [MASK] 处理
```

这种情况下，上下文信息太少，模型很难判断每个 `[MASK]` 原本是什么词。

因此，15% 可以理解为一种折中选择：

> 既能提供足够的预测目标，又不会过度破坏原始句子的上下文结构。

（5）被选中的 Token 一定会变成 `[MASK]` 吗

这里有一个容易误解的地方，并不是所有被选中用于预测的 Token 都会直接替换成 `[MASK]`。

在原始 BERT 中，会先随机选出 15% 的 Token 作为预测目标，然后对这些被选中的 Token 进行如下处理：

```
80%：替换成 [MASK]
10%：替换成随机 Token
10%：保持原 Token 不变
```

但无论输入中显示成什么形式，模型的训练目标都是一样的：

```
预测原始 Token：自然
```

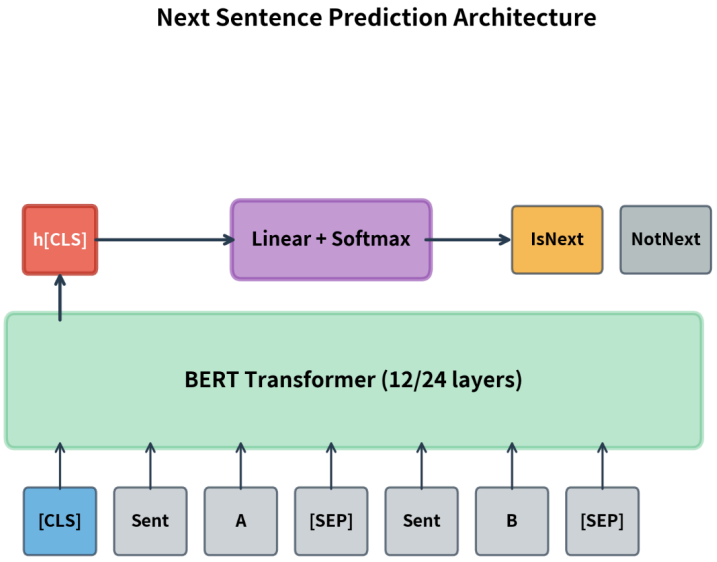
为什么要这样设计？因为真实下游任务中，输入文本通常不会出现 `[MASK]`。如果训练时所有预测位置都被替换成 `[MASK]`，模型可能过度依赖 `[MASK]` 这个特殊标记。

而加入随机替换和保持原词不变，可以让模型更适应真实文本输入，提高预训练和下游任务之间的一致性。

总结：

> MLM 通过随机遮住部分 Token，让 BERT 根据左右上下文预测原始 Token，从而学习 Token 级别的上下文语义表示。

2.4.3 预训练任务2：NSP（Next Sentence Prediction）



它是原始 BERT 中的第二个预训练任务，主要用来训练 BERT 理解**句子与句子之间的关系。**

（1）NSP 要解决什么问题

MLM 主要训练的是 Token 级别的上下文理解能力，但是很多 NLP 任务并不只关心单个句子内部的词语关系，还需要理解两个句子之间的关系。

例如：

```
问题：这家酒店的位置怎么样？
回答：离地铁站很近，出行比较方便。
```

模型需要判断这两个句子是否有关联。

再比如：

```
句子A：我今天去了图书馆。
句子B：在那里借了一本机器学习的书。
```

这两个句子之间存在明显的上下文衔接关系。

而下面这组句子：

```
句子A：我今天去了图书馆。
句子B：这家餐厅的牛排很好吃。
```

虽然两个句子本身都通顺，但它们之间没有明显的连续关系。

因此，BERT 需要一种任务来学习：

> 两个句子之间是否存在自然的上下文衔接关系。

这就是 NSP 任务的目的。

（2）NSP 的基本做法

NSP 的任务非常简单：

> 给 BERT 输入两个句子，让模型判断第二个句子是否是第一个句子的下一句。

它是一个二分类任务，标签只有两种：

```
IsNext：句子B是句子A的下一句
NotNext：句子B不是句子A的下一句
```

例如，正样本：

```
句子A：我今天去了图书馆。
句子B：在那里借了一本机器学习的书。
标签：IsNext
```

负样本：

```
句子A：我今天去了图书馆。
句子B：这家餐厅的牛排很好吃。
标签：NotNext
```

在构造训练数据时，正样本通常来自真实文本中连续出现的两个句子；负样本则是从语料中随机抽取另一个句子进行拼接。这样一来，不需要人工标注，模型也可以自动获得训练数据。所以，NSP 和 MLM 一样，也属于自监督学习任务。

（3）NSP 的输入格式

NSP 使用的是 BERT 的句子对输入格式：

```
[CLS] 句子A [SEP] 句子B [SEP]
```

例如：

```
[CLS] 我今天去了图书馆 [SEP] 在那里借了一本机器学习的书 [SEP]
```

其中：

```
[CLS]：放在整个输入序列的开头
[SEP]：用于分隔两个句子
Segment Embedding：用于区分句子A和句子B
```

这里正好对应前面讲过的 BERT 输入结构。

- 对于句子 A，Segment Embedding 通常记为 A；
- 对于句子 B，Segment Embedding 通常记为 B。

这样模型不仅知道输入了哪些 Token，也知道哪些 Token 属于第一个句子，哪些 Token 属于第二个句子。

（4）NSP 如何进行预测

BERT 编码完整个句子对之后，会输出每个 Token 对应的向量：

```
[CLS] → 向量
句子A中的Token → 向量
[SEP] → 向量
句子B中的Token → 向量
[SEP] → 向量
```

在 NSP 任务中，模型不会使用每个 Token 的输出向量分别判断，而是使用 `[CLS]` 位置的输出向量。

可以理解为：

```
[CLS] 输出向量
      ↓
分类层
      ↓
IsNext / NotNext
```

为什么使用 `[CLS]`？

因为 `[CLS]` 位于整个输入序列的开头，在 Transformer Encoder 的多层自注意力计算中，它可以和句子 A、句子 B 中的所有 Token 发生信息交互。

因此，经过多层编码后，`[CLS]` 的输出向量可以被看作整个句子对的综合表示。所以，在 NSP 任务中，BERT 使用 `[CLS]` 向量来判断两个句子之间的关系。

这也解释了为什么后续做文本分类、句子匹配等任务时，经常会使用 `[CLS]` 的输出向量作为整体文本表示。

2.4.4 MLM 与 NSP 的共同作用

BERT 的预训练阶段主要由两个任务组成：

```
MLM：掩码语言模型
NSP：下一句预测任务
```

这两个任务关注的层次不同。

- MLM 主要关注 Token 级别的语言理解

通过 MLM，BERT 可以学习到：Token 的含义不是固定的，而是会随着上下文变化。这也是 BERT 能够生成上下文词向量的关键原因。

- NSP 主要关注句子级别的关系理解，通过 NSP，BERT 可以学习到一定的句子间关系表示能力。

所以，MLM 和 NSP 的作用可以简单总结为：

```
MLM：让 BERT 学会理解词和上下文
NSP：让 BERT 学会理解句子和句子之间的关系
```

经过这两个预训练任务后，BERT 输出的向量就不再是随机向量，而是包含语言知识的上下文表示。